Davante

17

ENTORNOS DE DESARROLLO

Optimización del código

ÍNDICE

/ 1. Introducción y contextualización práctica 3

/ 2. Refactorización. Concepto y limitaciones 4

/ 3. Patrones de refactorización 4

/ 4. Caso práctico 1: “Renombrado de variables” 5

/ 5. Analizadores de código y pruebas 6

/ 6. Herramientas de ayuda a la refactorización 6

/ 7. Caso práctico 2: “Refactorización de jerarquía” 7

/ 8. Resumen y resolución del caso práctico de la unidad 8

/ 9. Bibliografía 9

© MEDAC 978-84-18983-00-9
Reservados todos los derechos. Queda rigurosamente prohibida, sin la autorización escrita de los titulares del copyright, bajo las sanciones establecidas en las leyes, la reproducción, transmisión y distribución total o parcial de esta obra por cualquier medio o procedimiento, incluidos la reprografía y el tratamiento informático.

OBJETIVOS

Conocer el concepto de refactorización del código.

Saber optimizar el código con refactorización.

Comprender las necesidades de modificar el código.

Saben utilizar herramientas de refactorización.

/ 1. Introducción y contextualización práctica

Una vez que disponemos del código fuente de nuestra aplicación, y éste, implementa toda la funcionalidad que se requiere, puede que su funcionamiento no se adapte a determinados estándares de calidad. Por ejemplo, puede que nuestra aplicación sea extremadamente lenta al realizar algún tipo de tarea. Esto suele suceder cuando realizamos una búsqueda en un conjunto de elementos. En otras ocasiones, puede que nuestra aplicación funcione correctamente, sin embargo, cada vez se vuelva más complicado poder añadir nuevas funcionalidades sin introducir errores en otras ya existentes. Para poder dar solución a este tipo de problemas podemos apoyarnos en técnicas que permiten optimizar el código mediante cambios en su estructura, organización, etc. Este proceso de cambios se conoce como **refactorización**. A través de este proceso es posible realizar mejoras en el código y optimizaciones que pueden mejorar incluso el rendimiento en tiempo de ejecución. Por ejemplo, supongamos que tenemos un dispositivo móvil que consume mucha batería cuando lanza nuestra aplicación. Una mejora de rendimiento podría ser consumir menos memoria y por lo tanto menos batería. En este tema veremos diferentes técnicas para poder realizar los cambios necesarios en el código, presentaremos herramientas para poder realizarla y veremos la relación que existe entre la refactorización y las pruebas. Escucha el siguiente audio en el que planteamos el caso práctico que iremos resolviendo a lo largo de esta unidad.



Fig. 1. La optimización de código requiere de amplia experiencia

Audio Intro. "Refactorización global"
http://bit.ly/2XobmCc

/ 2. Refactorización. Concepto y limitaciones

En el contexto de la programación y el diseño *software*, se conoce el concepto de **refactorización de código** como la **tarea que permite restructurar el código fuente cambiando la forma, pero sin cambiar el comportamiento de la funcionalidad**. Normalmente, la refactorización se utiliza para mejorar la estructura y diseño del código. Tras el proceso de refactorización es posible mejorar la comprensión del código y reducir su complejidad. De esta forma, es posible tener un **código más simple, limpio y mejor estructurado** desde el punto de vista arquitectónico y del modelo de objetos que estamos utilizando.

Otro aspecto que se ve favorecido tras un proceso de refactorización, es la **extensibilidad** del código. Esto quiere decir que, tras realizar un proceso de refactorización, es posible tener un código que es más fácil de extender en el futuro. No hay que olvidar que, uno de los **objetivos** de la refactorización es mejorar el **rendimiento** del *software*. Esto es posible porque tras el proceso de refactorización se puede dar que nuestro código requiera menos memoria para poder ser ejecutado.

Para poder realizar la refactorización se pueden aplicar una serie de **principios estandarizados** que permiten introducir unos **pequeños cambios en el código fuente que cambia la estructura**, pero preservan el comportamiento de la funcionalidad implementada. Muchos entornos de desarrollo permiten introducir automáticamente técnicas de refactorización. Hay que destacar que el proceso de refactorización puede ser muy delicado. Si realizamos el proceso correctamente ganaremos todas las ventajas de éste, sin embargo, si el proceso no es adecuado se pueden introducir nuevos fallos, perder funcionalidad, empeorar el rendimiento, etc. En ocasiones, el código fuente puede estar estructurado de tal forma que la refactorización puede ser muy complicada, incluso ineficiente. Por ello, es normal que, si existe un mal diseño anterior, aplicar técnicas de refactorización no sea una buena opción porque requiera más tiempo que empezar una nueva solución.



Audio 1. "Limitaciones de la refactorización"

http://bit.ly/3s8MDjd



/ 3. Patrones de refactorización

La refactorización es un proceso que requiere una serie de pasos que posteriormente necesitan un proceso de **pruebas para comprobar si ha cambiado la funcionalidad de los aspectos fundamentales del código**. Esta es la única forma en la que podemos asegurar que los cambios han sido correctos y seguros. Por esta razón, para poder realizar el proceso de refactorización de una forma más segura, existen unos **patrones ya preestablecidos que podemos aplicar** fácilmente:

- **Renombrado.** Este patrón se usa para cambiar el nombre de una clase, paquete...por otro que sea más representativo.
- **Sustitución de bloques.** Aplicando este patrón es posible sustituir un bloque del código por un método. De esta forma cada vez que utilicemos ese código solamente tendríamos que invocar al método. Gracias a este patrón podemos mejorar la reutilización de código.
- **Campos encapsulados.** Con este patrón es posible ocultar la información de asignación y consulta de las propiedades de una clase.
- **Movimiento de clases.** En muchas ocasiones es necesario mover una clase a otro paquete. Con este patrón se puede realizar el renombrado automáticamente en los paquetes afectados.
- **Extracción de la interfaz.** En ocasiones es necesario crear una interfaz con métodos públicos para poder proporcionarlos a otras clases.

- **Cambio de parámetros.** Es posible modificar el número de parámetros de un método y cambiar incluso su forma de acceso.
- **Mover a otro nivel.** Con este patrón podemos mover una clase a otro nivel de la jerarquía para que pueda incluso ser pública.
- **Borrado seguro.** En ocasiones es necesario borrar un elemento de código que puede estar siendo utilizado. Para realizar el borrado automáticamente (y de forma segura) deberíamos tener en cuenta dónde se está utilizando.



Vídeo 1. "Ejemplos de refactorización"
http://bit.ly/2m2mghD



/ 4. Caso práctico 1: "Renombrado de variables"

Planteamiento: En uno de nuestros actuales proyectos nos llega el siguiente código para poder continuar con una tarea de operaciones aritméticas. Antes de comenzar con el código vemos que existen algunos elementos que pueden dificultar su trabajo.

```
1.  double aux = (altura + anchura)*2;
2.  Console.WriteLine(aux);
3.  temp = altura * anchura;
4.  Console.WriteLine (aux);
```

Código 1. Renombrando variables.

Nudo: ¿Ves algún problema en el código? ¿Sería necesario refactorizar?

Desenlace: Según hemos visto en el tema, una de las características de la refactorización es poder cambiar el nombre de las variables para que puedan tener un significado coherente con el problema que estamos solucionando. En este caso se nos plantea la posibilidad de extender la funcionalidad que nos llega, pero vemos que las variables no tienen sentido. Una variable con el nombre *aux* y otra con el nombre *temp* no indican nada al programador y pueden ser fuente de posibles errores en el futuro. Por ello, lo que habría que hacer es cambiar los nombres de las variables utilizando los mecanismos que nos ofrezca el entorno de desarrollo para realizarlo de una manera segura. Podríamos hacerlo de forma manual, pero podríamos cometer más errores sin ser conscientes de ellos. A continuación, se muestra una propuesta para realizar el renombrado de las variables:

```
1.  double perimetro = (altura + anchura)*2;
2.  Console.WriteLine (perimetro);
3.  double area = altura * anchura;
4.  Console.WriteLine (area);
```

Código 2. Renombrando variables.

Como se puede observar, renombramos la variable aux como perímetro y temp como area. De esta forma estamos dando mayor sentido al código que vamos a escribir y en el futuro podrá ser fácilmente comprensible por el resto de los programadores que se incorporen al proyecto.

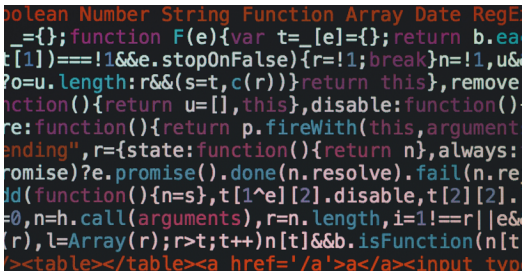


Fig. 2. Ejemplo de código que puede necesitar refactorización.

/ 5. Analizadores de código y pruebas

Analizadores de código

Los analizadores de código **permiten revisar el estilo, la calidad, el mantenimiento y el diseño del código fuente**, además de otras características que puedan ser necesarias. Se trata de herramientas que permiten garantizar determinadas métricas que podemos definir previamente. Generalmente, **un analizador** de código puede **detectar posibles infracciones** de las reglas que tiene configuradas y se indican a través del propio editor o con un informe al programador. En el caso de utilizar el editor, se puede indicar con otro color las reglas que han sido incumplidas. Cuando se detecta una regla que se incumple, **el editor puede proponer soluciones para arreglar estas situaciones.** Cuando utilizamos un entorno de desarrollo, normalmente los analizadores de código están integrados, y son sumamente importantes para poder realizar acciones rápidas y solucionar problemas típicos de una manera rápida y sencilla. Una vez que estamos acostumbrados a trabajar con el analizador de código y sus sugerencias, es muy complicado no tenerlo.

Refactorización y pruebas

Como hemos visto anteriormente, la refactorización requiere modificar el código fuente para poder llevar a cabo los cambios deseados. Cualquier cambio que hagamos en el código puede afectar a su comportamiento final y, por tanto, debe ser probado. **Existe una relación directa entre el proceso de refactorización y pruebas.** Por ello, es sumamente recomendable que **antes de realizar cualquier proceso de refactorización nos aseguremos que tenemos las pruebas necesarias para poder probar nuestra aplicación de manera automática**. Por ejemplo, sería adecuado tener pruebas automáticas para pruebas unitarias que permitan probar cada una de las funcionalidades por separado. Se puede utilizar un enfoque iterativo en el que realicemos pequeños cambios en el código que sean continuamente testeados.



Fig. 3. Los analizadores de código se basan en métricas

/ 6. Herramientas de ayuda a la refactorización

Existen diferentes tipos de técnicas que se pueden aplicar a la hora de realizar los cambios en el código fuente. Como hemos dicho anteriormente, muchos entornos de desarrollo proporcionan diferentes métodos para realizar esta tarea de refactorización.

Algunas de las **herramientas soportadas por los IDEs** son:

- **Mejora de la comprensión del código.** De esta forma es posible obtener una representación del código visual que permite ver las dependencias y facilitar tareas como la ingeniería inversa.

- **Mejora de la abstracción.** Permiten forzar la utilización de métodos setters y getters y generar código de mejor calidad utilizando polimorfismo.
- **Soporte para la división lógica del código.** De esta forma es posible dividir el código y reutilizar parte de éste en diferentes clases.
- **Soporte para nombres y localización del código.** Quizás es la forma más sencilla de refactorización, pues permite cambiar el nombre de variables, métodos, etc.

Existen muchas **herramientas** que permiten realizar la **refactorización de manera automática.** Algunas de ellas son:

- **Eclipse.** Al tratarse de una de los IDEs más utilizados, incorpora grandes funcionalidades para poder realizar la refactorización.
- **IntelliJ Idea y Netbeans.** Se trata de herramientas que permiten gestionar refactorización del código fácilmente.
- **Android Studio.** Al tratarse de un IDE para el desarrollo dispositivos móviles introduce grandes funcionalidades para la refactorización en diferentes lenguajes.



Fig. 4. Las herramientas de refactorización son muy potentes.

Vídeo 2. "Refactorización y generación de código"
https://bit.ly/3bfdEM2

/ 7. Caso práctico 2: "Refactorización de jerarquía"

Planteamiento: A nuestro compañero de equipo le llega el código fuente que se muestra a continuación:

```
1.  class Ejemplo{
2.      Coche[] coches;
3.  }
4.  class Coche {
5.  int id;
6.  boolean concursante (Ejemplo){
7.  for (int i = 0; i<e.coches.length; i++) {
8.      if (e.coches[i].id == id) return (true);
9.  }
10. return (false);
11. }
12. [...] if (x.participante(p)) [...]
```

Código 3. Refactorización de jerarquías.

Nudo: ¿Crees que el código es correcto? ¿Es necesario aplicar algún cambio?

Desenlace: En muchas ocasiones, a la hora de diseñar una jerarquía de clases en orientación objeto, es normal que la relación que exista entre las clases no quede suficientemente clara. Además, para poder realizar los cambios, cuando los detectamos, puede darse la situación de que sea realmente complicado. En el ejemplo anterior vemos que existe una relación entre la clase *Ejemplo* y la clase *Coche*, pero el *método concursante* no queda claro en qué clase debe de ir. Dado que no es una responsabilidad de la clase *Coche*, este método debería moverse a otra clase que tenga dicha responsabilidad. Como principio de la orientación objeto es importante destacar que una clase solo puede tener una responsabilidad, en caso contrario estaríamos incumpliendo los principios de orientación objeto como ***solid***. El resultado de realizar la refactorización de esta clase sería el siguiente:

```
1.  class Ejemplo{
2.    Coche[] coches;
3.    boolean concursantes (Coche x){
4.      for (int i=0; i<coches.length; i++) {
5.      if (coches [i].id == x.id.) return (true); }
6.      return (false);
7.      }
8.    }
9.
10. class Coche {
11. int id;
12. [...] if (e.participante(x))[...]
13. }
```

Código 4. Refactorización de jerarquías.

/ 8. Resumen y resolución del caso práctico de la unidad

En este tema hemos presentado los fundamentos necesarios para poder mejorar el código y realizar las optimizaciones necesarias que permitan llevar a cabo una correcta **refactorización.** Como hemos visto, existen diferentes técnicas y patrones para aplicar la refactorización con el fin de mejorar nuestro código y solucionar posibles problemas estructurales, de extensibilidad y mantenimiento en el futuro. Es importante destacar que, una vez que apliquemos la refactorización en el código, es necesario ejecutar un **conjunto de pruebas** que permitan garantizar que la funcionalidad básica no ha sido afectada y seguimos cumpliendo con todos los casos de usos y requisitos establecidos al comienzo. Existen **herramientas** que permiten realizar esas tareas automáticamente o bien, con gran ayuda para el desarrollador. También es importante saber identificar cuándo la refactorización no va a solucionar nuestros problemas y sería mejor comenzar una solución desde cero.

Resolución del caso práctico de la unidad

Como hemos visto en el tema, existen situaciones en las que añadir nuevas funcionalidades a un código existente puede ser muy complicado si éste no está organizado correctamente. Para ello podemos realizar algún proceso de refactorización y arreglar dichos problemas. Tal y como nos comentan en el denunciado, podemos intuir que existen algunas situaciones en las que podemos aplicar algunos patrones de refactorización. Si estamos utilizando un IDE de desarrollo, estos procesos deberían de ser suficientemente sencillos para aplicarlos rápidamente y proceder con los cambios que nos solicitan. Sin embargo, antes de realizar cualquier proceso de este tipo deberíamos solicitar permiso a la empresa asociada, pues una vez que realicemos los cambios deberemos realizar pruebas para asegurarnos de que nada ha sido destruido y que el comportamiento de la aplicación sigue siendo exactamente el mismo. Además, deberíamos establecer las responsabilidades para determinar qué sucede si algo falla en este proceso.

/ 9. Bibliografía

Fowler, M. & Beck, K. (2019). *Refactoring: improving the design of existing code*. Boston: Addison-Wesley.

Blanes, R. (2014). *El libro negro del programador: como conseguir una carrera de éxito desarrollando software y cómo evitar Los errores habituales*. Estados Unidos: CreateSpace.